VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

Railway Ticket Reservation System

Java Programming

Submitted by

Batch No. 12

Sri Charan 25881A05EC

Course Faculty

Ganesh Deshmukh

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

Table of Contents

Abstract . . . 2
1 Introduction . . . 3
2 Problem Statement . . . 3
3 Objectives . . . 3
4 Literature Review . . . 3
5 System Analysis . . . 4
5.1 Existing System . . . 4
5.2 Proposed System . . . 4
5.3 Scope . . . 4
6 Methodology . . . 4
6.1 System Modules . . . 4
6.2 System Flow . . . 5
6.3 Booking Algorithm . . . 5
6.4 Cancellation Algorithm . . . 5
6.5 Technology and Tools . . . 6
7 System Design . . . 6
7.1 Suggested Class Structure . . . 6
7.2 Data Validation and Error Handling . . . 6
8 Project Implementation Code . . . 6
8.1 Source Code . . . 6
8.2 Compilation and Execution . . . 9
9 Results and Discussion . . . 10
9.1 Program Execution Screenshots / Actual Results . . . 10
10 Mapping POs and SDGs . . . 12
10.1 Program Outcomes (POs) Mapping . . . 12
10.2 Sustainable Development Goals (SDGs) Mapping . . . 12
11 Limitations and Future Enhancements . . . 12
12 Conclusion . . . 13
Bibliography . . . 14

Abstract

Railway transportation serves a large number of passengers, and reservation services must manage train information, passenger details, seat availability, bookings, and cancellations in an organised manner. This project presents a Java-based Railway Ticket Reservation System that models the main operations involved in reserving a railway ticket. The system is designed around a menu-driven interface through which a user can view available trains, enter passenger information, book a ticket, view booking details, and cancel a reservation. Java classes and methods are used to separate train, passenger, and reservation responsibilities, while conditional statements, loops, collections, and input validation support the application workflow.

The proposed system demonstrates how object-oriented programming can be applied to a practical reservation problem. A unique booking identifier can be assigned to each successful reservation, and seat availability can be updated after booking or cancellation. The project is an educational prototype and does not connect to the official railway reservation network or process real payments. Persistent storage, authentication, concurrent booking control, and a graphical or web interface are possible future extensions. **Keywords:** Java, railway reservation, ticket booking, object-oriented programming, passenger details, seat availability.

1 Introduction

Railway ticket reservation involves several connected tasks, including searching for trains, checking available seats, recording passenger details, confirming bookings, and processing cancellations. Handling these tasks manually can be slow and can lead to inconsistent records. A software-based reservation system helps organise the information and provides a consistent sequence of operations.

The Railway Ticket Reservation System in this report is a Java programming project intended to demonstrate these operations in a simplified environment. It applies fundamental Java concepts such as classes, objects, methods, collections, decision-making, iteration, and exception handling. A menu-driven console interface keeps the application easy to use and suitable for demonstrating the underlying program logic.

2 Problem Statement

A reservation workflow needs to maintain train and passenger information and ensure that a booking is accepted only when seats are available. It must also allow users to retrieve a booking and cancel it without losing track of seat availability. The project addresses this problem by proposing a small application that performs these operations in a structured and understandable way.

3 Objectives

- To design a simple Java application for railway ticket reservation.
- To display train details and the availability of seats.
- To collect and validate passenger information before booking.
- To generate a unique identifier for each reservation.
- To provide booking lookup and ticket cancellation operations.
- To update seat availability consistently after booking and cancellation.
- To demonstrate object-oriented programming and basic data handling in Java.

4 Literature Review

Reservation systems commonly use structured records to represent services, customers, and transactions. A booking workflow typically checks availability before creating a reservation and updates the relevant inventory after a successful transaction. Cancellation reverses the reservation and restores the available capacity, subject to the system's rules.

Java is suitable for an educational implementation because it supports object-oriented design, type checking, exception handling, and standard collection classes. A console-based prototype can demonstrate the core logic without requiring a database or network connection. Production railway systems require additional capabilities such as secure user accounts, payment integration,

persistent databases, transaction management, audit trails, and protection against simultaneous bookings. These features are outside the scope of this simplified project.

5 System Analysis

5.1 Existing System

In a manual or poorly coordinated process, train and passenger records may be maintained separately, and checking availability or locating an existing booking may require additional effort. Errors can also occur if a cancellation is not reflected in the seat count.

5.2 Proposed System

The proposed application presents a single menu for train listing, ticket booking, booking search, cancellation, and exit. It keeps reservation data in structured program objects during execution and applies validation before accepting user input. In a basic in-memory implementation, data is available only while the program is running; persistent storage is recommended for a real deployment.

5.3 Scope

The project is intended for academic demonstration. It covers the basic reservation lifecycle and does not claim to provide official railway data, live train schedules, real seat allocation, online payments, or integration with Indian Railways services.

6 Methodology

6.1 System Modules

- **Train Management:** Stores train number, train name, source, destination, and available seats.
- **Passenger Management:** Captures essential passenger details required for a reservation.
- **Ticket Booking:** Checks seat availability, records the passenger, assigns a booking ID, and reduces available seats.
- **Booking Search:** Finds a reservation using its booking ID and displays the associated details.
- **Cancellation:** Verifies the booking, marks it cancelled, and restores the seat count once.
- **Menu and Validation:** Presents available actions and handles invalid choices or incomplete input.

6.2 System Flow

Start application
Display menu: View trains / Book ticket / Search booking / Cancel ticket / Exit
Read and validate the selected option
Perform the selected operation and update reservation data if required
Display confirmation or an error message
Return to menu until Exit is selected
End

Figure 1: High-level workflow of the Railway Ticket Reservation System

6.3 Booking Algorithm

1. Display the list of trains and their available seats.
2. Ask the user to select a valid train.
3. Verify that at least one seat is available.
4. Read passenger details and validate the input.
5. Create a reservation record and assign a unique booking ID.
6. Reduce the selected train's available-seat count by one.
7. Display the booking confirmation and booking ID.

6.4 Cancellation Algorithm

1. Request the booking ID from the user.
2. Search for the corresponding reservation.
3. If no reservation exists, display a suitable message.
4. If the booking is already cancelled, do not change the seat count again.
5. Otherwise, mark the booking as cancelled and restore one seat to the train.
6. Display the cancellation status.

6.5 Technology and Tools

- **Programming language:** Java.
- **Development environment:** Any Java-compatible IDE or command-line JDK.
- **Interface:** Menu-driven console interface.
- **Data handling:** Java classes and in-memory collections for the prototype.

7 System Design

7.1 Suggested Class Structure

- **Train:** Contains train number, name, source, destination, and available-seat count.
- **Passenger:** Contains passenger name and other required passenger details.
- **Reservation:** Contains booking ID, train reference, passenger information, and booking status.
- **ReservationSystem:** Coordinates train listing, booking, lookup, cancellation, and menu operations.
- **Main:** Starts the application and runs the main menu loop.

7.2 Data Validation and Error Handling

The application should reject invalid menu selections, unknown train numbers, blank mandatory fields, and booking requests when no seats are available. Numeric input should be checked before conversion to avoid program termination due to malformed input. Cancellation must be idempotent: a booking that has already been cancelled must not restore another seat on a repeated cancellation attempt.

8 Project Implementation Code

The following Java program implements the main features of the proposed console-based prototype: listing sample trains, booking a ticket, generating a booking ID, searching for a booking, cancelling a booking, validating numeric input, and restoring a seat only once after cancellation. The sample train information is illustrative and is not connected to live railway services. Save the code as `RailwayTicketReservationSystem.java`, compile it with a Java Development Kit, and run the class from a terminal or IDE.

8.1 Source Code

Listing 1: Railway Ticket Reservation System implementation

```
1 import java.util.ArrayList;
2 import java.util.LinkedHashMap;
3 import java.util.List;
4 import java.util.Map;
```

```java
import java.util.Scanner;

class Train {
    private final int number;
    private final String name;
    private final String source;
    private final String destination;
    private int availableSeats;

    Train(int number, String name, String source, String destination, int seats) {
        this.number = number;
        this.name = name;
        this.source = source;
        this.destination = destination;
        this.availableSeats = seats;
    }

    int getNumber() { return number; }
    int getAvailableSeats() { return availableSeats; }
    void reserveSeat() { availableSeats--; }
    void releaseSeat() { availableSeats++; }

    @Override
    public String toString() {
        return number + " | " + name + " | " + source + " -> " + destination
               + " | Seats available: " + availableSeats;
    }
}

class Reservation {
    private final String bookingId;
    private final String passengerName;
    private final int age;
    private final Train train;
    private boolean cancelled;

    Reservation(String bookingId, String passengerName, int age, Train train) {
        this.bookingId = bookingId;
        this.passengerName = passengerName;
        this.age = age;
        this.train = train;
        this.cancelled = false;
    }

    String getBookingId() { return bookingId; }
    boolean isCancelled() { return cancelled; }
    Train trainReference() { return train; }
    void cancel() { cancelled = true; }

    void display() {
        System.out.println("Booking ID : " + bookingId);
        System.out.println("Passenger : " + passengerName);
        System.out.println("Age : " + age);
        System.out.println("Train : " + train);
        System.out.println("Status : " + (cancelled ? "CANCELLED" : "CONFIRMED"));
    }
}

public class RailwayTicketReservationSystem {
    private static final Scanner scanner = new Scanner(System.in);
    private static final List<Train> trains = new ArrayList<>();
    private static final Map<String, Reservation> bookings = new LinkedHashMap<>();
    private static int nextBookingNumber = 1001;

```

```
    public static void main(String[] args) {
        loadSampleTrains();
        boolean running = true;
        while (running) {
            System.out.println("\n=== RAILWAY TICKET RESERVATION SYSTEM ===");
            System.out.println("1. View trains");
            System.out.println("2. Book ticket");
            System.out.println("3. Search booking");
            System.out.println("4. Cancel ticket");
            System.out.println("5. Exit");
            int choice = readInt("Enter choice: ");
            switch (choice) {
                case 1: viewTrains(); break;
                case 2: bookTicket(); break;
                case 3: searchBooking(); break;
                case 4: cancelTicket(); break;
                case 5: running = false; System.out.println("Thank you."); break;
                default: System.out.println("Invalid choice. Select 1 to 5.");
            }
        }
        scanner.close();
    }

    private static void loadSampleTrains() {
        trains.add(new Train(12723, "Telangana Express", "Hyderabad", "New Delhi", 3));
        trains.add(new Train(12702, "Hussain Sagar Express", "Hyderabad", "Mumbai", 2));
        trains.add(new Train(12603, "Charminar Express", "Hyderabad", "Chennai", 4));
    }

    private static void viewTrains() {
        System.out.println("\nTrain No. | Name | Route | Availability");
        for (Train train : trains) System.out.println(train);
    }

    private static void bookTicket() {
        viewTrains();
        int trainNumber = readInt("Enter train number: ");
        Train selected = findTrain(trainNumber);
        if (selected == null) {
            System.out.println("Train not found.");
            return;
        }
        if (selected.getAvailableSeats() <= 0) {
            System.out.println("No seats available on this train.");
            return;
        }
        String name;
        do {
            System.out.print("Enter passenger name: ");
            name = scanner.nextLine().trim();
            if (name.isEmpty()) System.out.println("Name cannot be blank.");
        } while (name.isEmpty());
        int age = readInt("Enter passenger age: ");
        if (age < 1 || age > 120) {
            System.out.println("Enter a valid age between 1 and 120.");
            return;
        }
        String id = "RTS" + nextBookingNumber++;
        Reservation reservation = new Reservation(id, name, age, selected);
        bookings.put(id, reservation);
        selected.reserveSeat();
        System.out.println("Booking successful. Your booking ID is " + id);
        reservation.display();
    }
```

```
133
134     private static void searchBooking() {
135         System.out.print("Enter booking ID: ");
136         String id = scanner.nextLine().trim().toUpperCase();
137         Reservation reservation = bookings.get(id);
138         if (reservation == null) System.out.println("Booking not found.");
139         else reservation.display();
140     }
141
142     private static void cancelTicket() {
143         System.out.print("Enter booking ID to cancel: ");
144         String id = scanner.nextLine().trim().toUpperCase();
145         Reservation reservation = bookings.get(id);
146         if (reservation == null) {
147             System.out.println("Booking not found.");
148         } else if (reservation.isCancelled()) {
149             System.out.println("This booking has already been cancelled.");
150         } else {
151             reservation.cancel();
152             // Restore the seat only once for an active reservation.
153             findTrainForReservation(reservation).releaseSeat();
154             System.out.println("Booking " + id + " cancelled successfully.");
155         }
156     }
157
158     private static Train findTrainForReservation(Reservation reservation) {
159         // Each reservation retains its train object; this helper resolves it by listing.
160         // In this small prototype, the train reference is included in the reservation.
161         return reservation.trainReference();
162     }
163
164     private static Train findTrain(int number) {
165         for (Train train : trains) if (train.getNumber() == number) return train;
166         return null;
167     }
168
169     private static int readInt(String prompt) {
170         while (true) {
171             System.out.print(prompt);
172             String input = scanner.nextLine().trim();
173             try {
174                 return Integer.parseInt(input);
175             } catch (NumberFormatException ex) {
176                 System.out.println("Please enter a whole number.");
177             }
178         }
179     }
180 }
```

8.2 Compilation and Execution

1. Install a Java Development Kit (JDK).
2. Save the source as `RailwayTicketReservationSystem.java`.
3. Open a terminal in the source-code directory and run `javac RailwayTicketReservationSystem.java`.
4. Start the program using `java RailwayTicketReservationSystem`.
5. Select a menu option and follow the prompts to view trains, book a ticket, search by booking ID, or cancel a reservation.

The implementation stores data in memory for demonstration. Bookings are cleared when the program exits; a database would be needed for persistent records.

9 Results and Discussion

The design provides a clear workflow for the essential operations of a reservation application. Viewing trains helps users compare the available options; booking creates a reservation only after checking capacity; booking lookup retrieves stored details; and cancellation updates the reservation status and seat count. These operations demonstrate the use of objects, methods, collections, conditional logic, loops, and input validation in Java.

The expected functional checks for the prototype are listed below. Actual results should be confirmed by running the implementation in the chosen development environment.

Test case	**Action**	**Expected result**
View trains	Select the train-list option	Train details and seat counts are displayed
Successful booking	Select a train with seats and enter valid details	Booking ID is generated and seats decrease by one
No availability	Attempt booking when seats equal zero	Booking is rejected with a clear message
Booking search	Enter a valid booking ID	Matching reservation details are displayed
Invalid booking ID	Search for an unknown ID	A not-found message is displayed
Cancellation	Cancel an active booking	Status changes to cancelled and a seat is restored
Repeated cancellation	Cancel the same booking again	No additional seat is restored

9.1 Program Execution Screenshots / Actual Results

Complete this subsection after compiling and running the Java program. Replace each bordered placeholder with a screenshot of the actual console output. The screenshots should show the program running in your IDE or terminal. An in-memory implementation is appropriate for demonstrating program flow, but its records are lost when the application closes. For a practical deployment, records should be stored in a database and booking operations should use transactions to prevent inconsistent seat counts when multiple users act simultaneously.

```
PS C:\Users\DELL> notepad RailwayTicketReservationSystem.java
PS C:\Users\DELL> javac RailwayTicketReservationSystem.java
PS C:\Users\DELL> java RailwayTicketReservationSystem

=== RAILWAY TICKET RESERVATION SYSTEM ===
1. View trains
2. Book ticket
3. Search booking
4. Cancel ticket
5. Exit
Enter choice:
```

Figure 1: Successful main list output

```
=== RAILWAY TICKET RESERVATION SYSTEM ===
1. View trains
2. Book ticket
3. Search booking
4. Cancel ticket
5. Exit
Enter choice: 2

Train No. | Name | Route | Availability
12723 | Telangana Express | Hyderabad -> New Delhi | Seats available: 3
12702 | Hussain Sagar Express | Hyderabad -> Mumbai | Seats available: 2
12603 | Charminar Express | Hyderabad -> Chennai | Seats available: 4
Enter train number: 12723
Enter passenger name: Sri Charan M
Enter passenger age: 19
Booking successful. Your booking ID is RTS1001
Booking ID : RTS1001
Passenger  : Sri Charan M
Age        : 19
Train      : 12723 | Telangana Express | Hyderabad -> New Delhi | Seats available: 2
Status     : CONFIRMED
```

Figure 2: Successful ticket booking output

```
=== RAILWAY TICKET RESERVATION SYSTEM ===
1. View trains
2. Book ticket
3. Search booking
4. Cancel ticket
5. Exit
Enter choice: 3
Enter booking ID: RTS1001
Booking ID : RTS1001
Passenger  : Sri Charan M
Age        : 19
Train      : 12723 | Telangana Express | Hyderabad -> New Delhi | Seats available: 2
Status     : CONFIRMED
```

Figure 3: Successful ticket booked or cancelletion output

10 Mapping POs and SDGs

10.1 Program Outcomes (POs) Mapping

PO No.	Program Outcome	Relevance to Project
PO1	Engineering Knowledge	Applies Java programming fundamentals and data structures.
PO2	Problem Analysis	Identifies the need to coordinate availability, bookings, and cancellations.
PO3	Design/Development of Solutions	Organises reservation operations into separate classes and modules.
PO5	Modern Tool Usage	Uses a Java development environment and standard libraries.
PO9	Individual and Team Work	Can be developed, tested, and documented through organised project work.

10.2 Sustainable Development Goals (SDGs) Mapping

SDG No.	Goal	Relevance to Project
SDG 9	Industry, Innovation and Infrastructure	Demonstrates a digital approach to organising reservation information.
SDG 11	Sustainable Cities and Communities	Supports the study of efficient public-transport service workflows.
SDG 12	Responsible Consumption and Production	Encourages electronic record handling rather than unnecessary paper records in the proposed workflow.

11 Limitations and Future Enhancements

The current project is an educational model and has several limitations:

- It does not connect to official railway schedules, fares, or live seat inventory.
- In-memory records are not retained after the program terminates.
- It does not include user authentication, payment processing, or a graphical interface.
- Simultaneous booking requests and production-level security are not handled.

Future enhancements may include database integration using JDBC and MySQL, a graphical interface using Java Swing or JavaFX, secure login, ticket printing, email or SMS notifications, search by journey date, and transaction-safe seat allocation. Integration with external railway services would require authorised APIs and appropriate security controls.

12 Conclusion

The Railway Ticket Reservation System demonstrates how Java can be used to model a common reservation workflow. By separating train, passenger, and reservation data and providing booking, lookup, and cancellation operations, the project applies core object-oriented programming concepts to a practical problem. The design also highlights the importance of input validation and consistent seat management. Database persistence, a user-friendly interface, security, and authorised live-service integration would be necessary to evolve the prototype into a more complete application.

Bibliography

[1] Oracle, *Java Documentation*. Available at: `https://docs.oracle.com/en/java/`.

[2] Oracle, *Collections Framework Overview*. Available at: `https://docs.oracle.com/javase/`.

[3] Oracle, *JDBC Basics*. Available at: `https://docs.oracle.com/javase/tutorial/jdbc/`.

[4] Herbert Schildt, *Java: The Complete Reference*, McGraw Hill.